



Banking Transactions — User Guide

A screenshot-driven walkthrough of the Homework 1 Banking Transactions application: how to run it, what it can do, and how every feature looks in practice.

The project is a complete banking-transactions system built **test-first (TDD)**. Its defining trait: **one REST contract**, implemented by **four interchangeable backends** and consumed by **three interchangeable frontends** — mix and match any pair. This guide demonstrates two of each, live.

1. What's in the box

Layer	Options	Location
Backends (pick one · all on :3000 · identical contract & tests)	Java + Spring Boot · Go + Chi · Python + FastAPI · C# / .NET	backend/ , backend-go/ , backend-fastapi/ , backend-dotnet/
Frontends (pick one)	Angular + Microsoft Fluent (web) · Electron (desktop) · Flutter Web	frontend/ , frontend-electron/ , frontend-flutter/

Features (every backend implements all): create / list / fetch transactions with structured validation errors; filter history by account, type and date range; account **balance**, **summary** and **simple-interest**; **CSV export**; per-IP **rate limiting**; interactive API docs + health endpoint.

2. How to run it

From the `homework-1/` folder, run the interactive launcher:

```
./run.sh
```

Choose a **backend** (Java / Go / FastAPI / .NET) and a **frontend** (Angular / Electron / Flutter). It then starts the backend on `http://localhost:3000`, waits until healthy and **seeds sample data** (`demo/seed.sh`), launches the frontend, and stops everything on **Ctrl+C**. You only need the toolchain for the stack you pick.

```
# Python + FastAPI backend
cd backend-fastapi
python3 -m venv .venv && ./venv/bin/pip install -r requirements.txt
./venv/bin/uvicorn app.main:app --port 3000

# Go + Chi backend
cd backend-go && go run ./cmd/server          # listens on :3000

# Angular frontend (proxies /api -> :3000)
cd frontend && npm install && npm start        # http://localhost:4200

# Electron desktop frontend
cd frontend-electron && npm install && npm start
```

The **API-online** chip in the top-right of any frontend confirms it reached the backend.

3. Mix & match: two frontends × two backends (verified live)

The screenshots in this guide were captured against **two different stacks**, proving the shared contract works regardless of the combination:

Combination	Frontend	Backend
Stack A	Electron (desktop)	Python · FastAPI
Stack B	Angular · Fluent (web)	Go · Chi

Backend #1 Python · FastAPI
frontend: Electron (desktop)
\$./run.sh # choose 3) FastAPI, then 2) Electron
venv + deps installed
Starting backend: Python + FastAPI
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:3000
\$ curl -s :3000/openapi.json | jq .info
{ "title": "Banking Transactions API (FastAPI)",
"version": "v1" }
interactive Swagger UI → :3000/docs

Backend #2 Go · Chi
frontend: Angular (web :4200)
\$./run.sh # choose 2) Go, then 1) Angular
Starting backend: Go + Chi
2026/07/03 14:15:29 Banking API listening on :3000
Seeding sample data... done
\$ curl -s :3000/transactions | jq length
8 # same contract, same seed data
\$ curl -s :3000/accounts/ACC-12345/balance
{ "accountId":"ACC-12345", "balance":1525.25 }

Also available (same contract, same tests): Java + Spring Boot, C# / .NET backends; Flutter Web frontend. Every backend passes the identical TDD suite — see each backend - */README.md.

Both backends booted on :3000 and served the same seeded data.

4. Stack B — Angular (Fluent UI) on the Go backend

Banking

Transactions

New transaction

Account tools

Banking Transactions

API online

API docs

Dark

Transactions

Export CSV

New transaction

ACCOUNT

TYPE

FROM

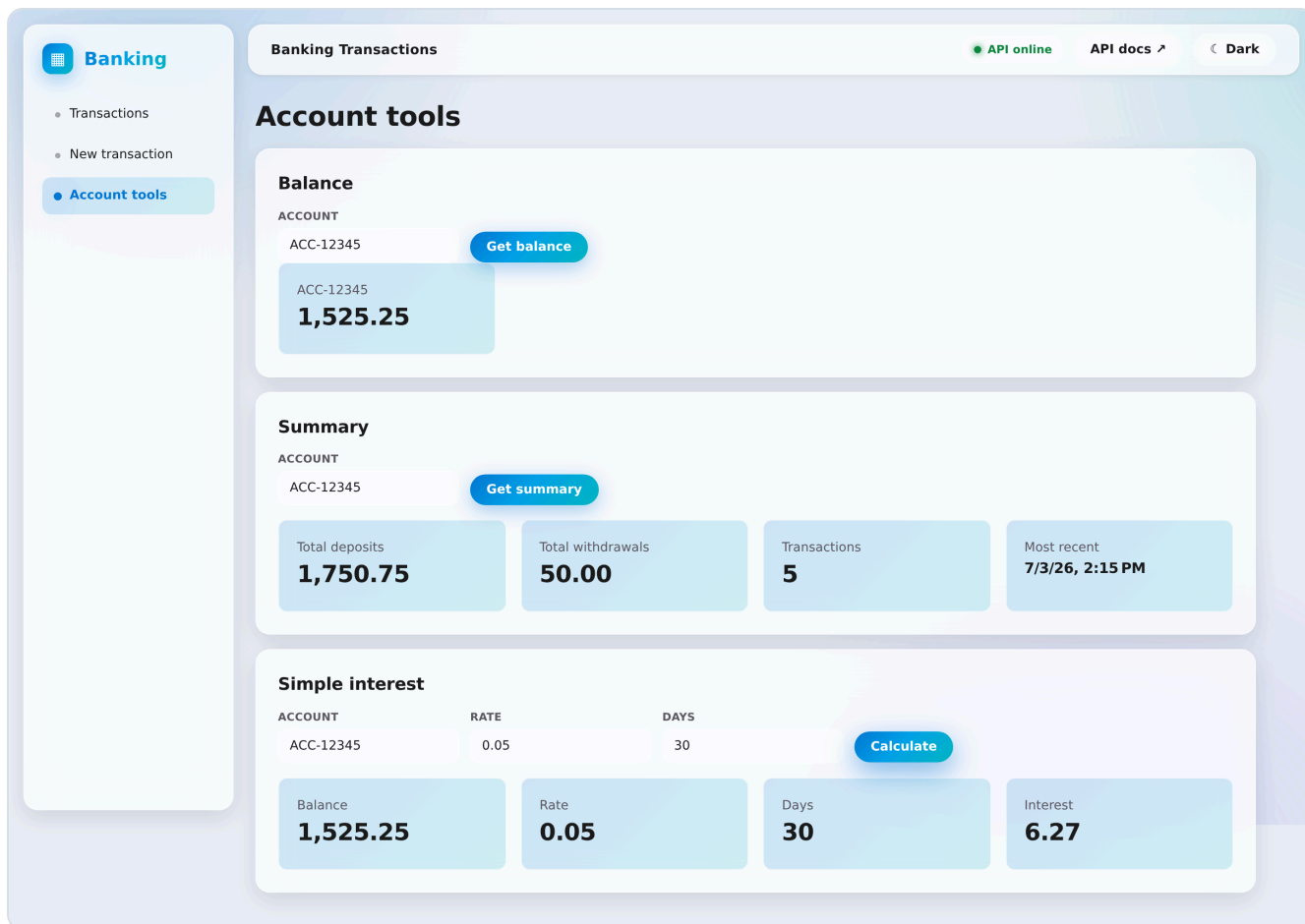
TO

Apply

Clear

ID	FROM	TO	AMOUNT	TYPE	STATUS	DATE
83afe3b5...	ACC-00000	ACC-12345	500.00 USD	Deposit	Completed	7/3/26, 2:15 PM
5d296e2b...	ACC-00000	ACC-12345	1,250.75 USD	Deposit	Completed	7/3/26, 2:15 PM
8a2186f7...	ACC-12345	ACC-67890	100.50 USD	Transfer	Completed	7/3/26, 2:15 PM
13d87ef5...	ACC-12345	ACC-99001	75.00 USD	Transfer	Completed	7/3/26, 2:15 PM
f9d0e93d...	ACC-12345	ACC-00000	50.00 USD	Withdrawal	Completed	7/3/26, 2:15 PM
efe2b891...	ACC-00000	ACC-67890	900.00 EUR	Deposit	Completed	7/3/26, 2:15 PM
1980b6ab...	ACC-67890	ACC-55512	300.00 EUR	Transfer	Completed	7/3/26, 2:15 PM
e96b9252...	ACC-55512	ACC-00000	120.25 GBP	Withdrawal	Completed	7/3/26, 2:15 PM

Angular transactions list — served by the Go backend.



Angular account tools: balance, summary and interest, computed by Go.

5. Browsing transactions

Screenshots below: Electron desktop app on the FastAPI backend.

The **Transactions** view lists every transaction with short ID, accounts, amount + currency, type, a colour-coded **status** badge, and date. The filter bar supports **Account** (sender or receiver), **Type**, and a **From/To** date range — combinable with logical AND. **↓ Export CSV** downloads the list.

Banking

• Transactions

• New transaction

• Account tools

Banking Transactions · Desktop

API online

Light

Transactions

Export CSV

New transaction

ACCOUNT

ACC-12345

TYPE

All

FROM

mm/dd/yyyy

TO

mm/dd/yyyy

Apply

Clear

ID	FROM	TO	AMOUNT	TYPE	STATUS	DATE
12404d0e...	ACC-00000	ACC-12345	500.00 USD	Deposit	Completed	7/3/2026, 2:02:48 PM
e62399e8...	ACC-00000	ACC-12345	1250.75 USD	Deposit	Completed	7/3/2026, 2:02:48 PM
d8fb9852...	ACC-12345	ACC-67890	100.50 USD	Transfer	Completed	7/3/2026, 2:02:48 PM
fae83cd9...	ACC-12345	ACC-99001	75.00 USD	Transfer	Completed	7/3/2026, 2:02:48 PM
5f123ec7...	ACC-12345	ACC-00000	50.00 USD	Withdrawal	Completed	7/3/2026, 2:02:48 PM
21110ddf...	ACC-00000	ACC-67890	900.00 EUR	Deposit	Completed	7/3/2026, 2:02:48 PM
8e9440f1...	ACC-67890	ACC-55512	300.00 EUR	Transfer	Completed	7/3/2026, 2:02:48 PM
8667c50a...	ACC-55512	ACC-00000	120.25 GBP	Withdrawal	Completed	7/3/2026, 2:02:48 PM

Transactions list with filters and status badges.

6. Creating a transaction

Click **+ New transaction**, fill the fields, press **Create transaction**. The form validates input and shows a message **under each field** when something is wrong.

- **From / To account** — must match `ACC-XXXXX` .
- **Amount** — positive number, max 2 decimals.
- **Currency** — valid ISO 4217 code (USD, EUR, GBP, JPY).
- **Type** — deposit, withdrawal, or transfer.

Banking

Transactions

New transaction

Account tools

Banking Transactions · Desktop

API onlineLight

New transaction

FROM ACCOUNT

BADACC

String should match pattern '^ACC-[A-Za-z0-9]+\$'

TO ACCOUNT

ACC-67890

AMOUNT

-10

Input should be greater than 0

CURRENCY

USD

TYPE

Transfer

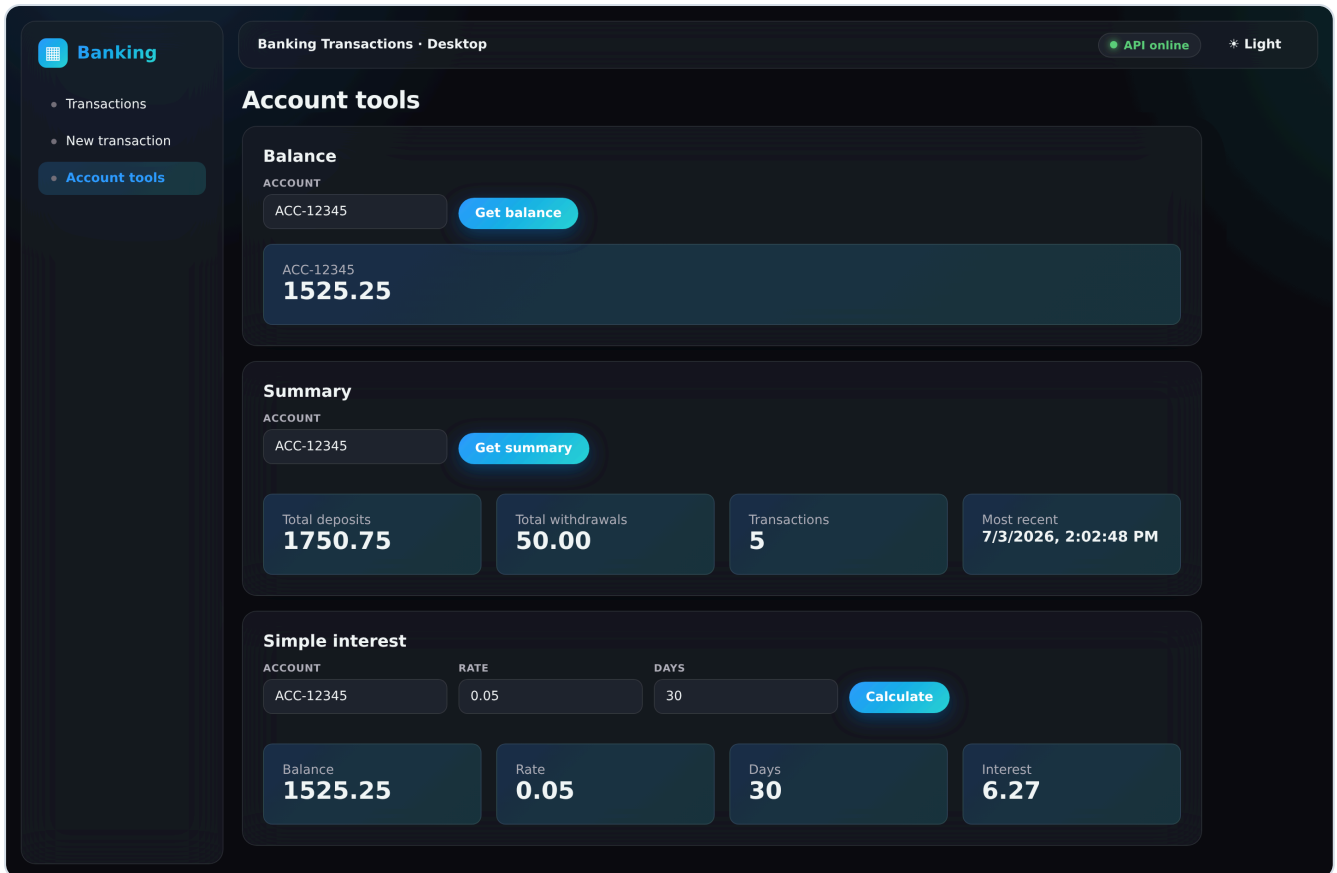
Create transaction

Cancel

New-transaction form showing per-field validation errors.

7. Account tools

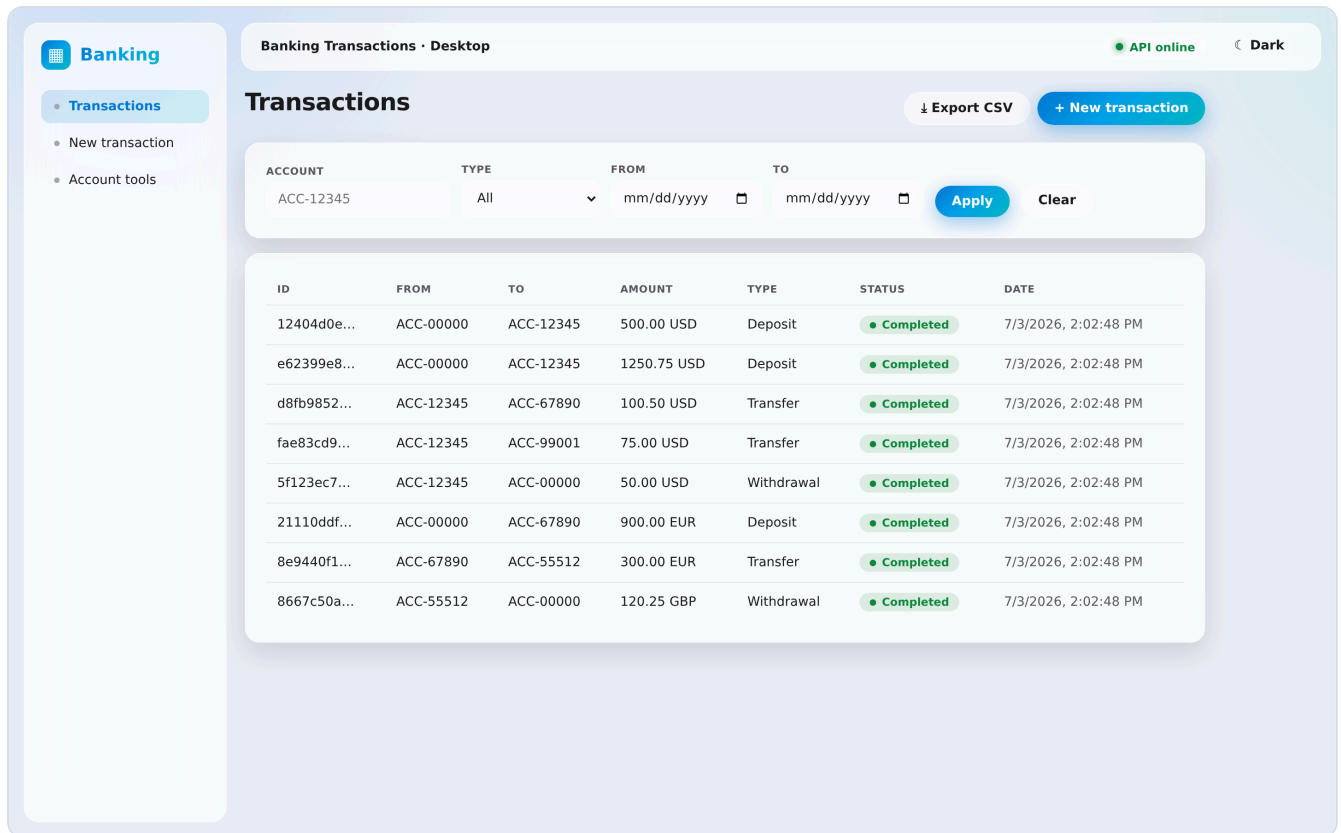
Enter an account number and click the button in each card: **Balance** (from completed transactions), **Summary** (deposits / withdrawals / count / most-recent date), and **Simple interest** ($\text{balance} \times \text{rate} \times \text{days} \div 365$, 2 decimals).



Balance, summary and simple-interest results.

8. Light & dark themes

Use the ✱ **Light** / ☾ **Dark** toggle in the top-right — tables, badges and cards all adapt.



Transactions list in light theme.

9. The REST API

The UI is a thin client over a REST API you can call directly with `curl`, Postman, or the VS Code REST Client. Interactive Swagger UI is at `/docs`, the OpenAPI spec at `/openapi.json`.

Method	Endpoint	Purpose
POST	<code>/transactions</code>	Create (201; 400 with structured error list).
GET	<code>/transactions</code>	List with accountId / type / from / to filters.
GET	<code>/transactions/{id}</code>	Fetch one (404 if missing).
GET	<code>/transactions/export?format=csv</code>	Download all as CSV.
GET	<code>/accounts/{id}/balance</code>	Account balance.
GET	<code>/accounts/{id}/summary</code>	Deposits / withdrawals / count / date.
GET	<code>/accounts/{id}/interest?rate=&days=</code>	Simple interest.



Banking Transactions API — Reference

Homework 1 · FastAPI implementation · base URL `http://localhost:3000` · live request/response samples

POST /transactions

201 Created

Create a transaction. Server sets id, timestamp, status.

REQUEST

```
curl -X POST http://localhost:3000/transactions \
-H "Content-Type: application/json" \
-d '{"fromAccount": "ACC-12345", "toAccount": "ACC-67890",
    "amount": 100.50, "currency": "USD", "type": "transfer"}'
```

RESPONSE

```
{
  "id": "e8436d57-513b-491f-9e86-e2bc49afd361",
  "fromAccount": "ACC-12345",
  "toAccount": "ACC-67890",
  "amount": 100.5,
  "currency": "USD",
  "type": "transfer",
  "timestamp": "2026-07-03T14:06:32.858715Z",
  "status": "completed"
}
```

GET /accounts/{id}/balance

200 OK

Balance from completed transactions (deposit / withdrawal / transfer rules).

REQUEST

```
curl http://localhost:3000/accounts/ACC-12345/balance
```

RESPONSE

```
{ "accountId": "ACC-12345", "balance": 1424.75 }
```

GET /accounts/{id}/summary

200 OK

Totals + count + most-recent date for an account.

RESPONSE

```
{
  "totalDeposits": 1750.75,
  "totalWithdrawals": 50.0,
  "transactionCount": 6,
  "mostRecentTransactionDate": "2026-07-03T14:06:32Z"
}
```

POST /transactions

400 Bad Request

Invalid input returns a structured error list (bad account format, non-positive amount).

RESPONSE

```
{
  "error": "Validation failed",
  "details": [
    { "field": "fromAccount",
      "message": "account must match ACC-XXXX" },
    { "field": "amount",
      "message": "amount must be a positive number" }
  ]
}
```

GET /accounts/{id}/interest

200 OK

Simple interest = balance × rate × days ÷ 365 (2dp).

REQUEST

```
curl "http://localhost:3000/accounts/ACC-12345/interest?rate=0.05&days=30"
```

RESPONSE

```
{
  "accountId": "ACC-12345", "balance": 1424.75,
  "rate": 0.05, "days": 30, "interest": 5.86
}
```

GET /transactions?type=transfer

200 OK

List with filters: accountId, type, from/to date range (combinable).

RESPONSE (TRUNCATED)

```
[
  {
    "id": "d8fb9852-145a-4f55-...",
    "fromAccount": "ACC-12345",
    "toAccount": "ACC-67890",
    "amount": 100.5,
    "currency": "USD",
    "type": "transfer",
    "status": "completed"
  }, ...
]
```

GET /transactions/export?format=csv

200 OK

Exports all transactions as text/csv (RFC 4180) with a download header.

RESPONSE

```
id,fromAccount,toAccount,amount,currency,type,timestamp,status
d8fb...,ACC-12345,ACC-67890,100.50,USD,transfer,2026-07-03T14:02:48Z,completed
fae8...,ACC-12345,ACC-99001,75.00,USD,transfer,2026-07-03T14:02:48Z,completed
...
```

Also available: interactive Swagger UI at `/docs` and OpenAPI spec at `/openapi.json`.

REST API endpoints with live request/response samples.